

Лабораторная работа №2. HTTP-методы: обработка GET, POST, PUT, DELETE

Краткое содержание лабораторной работы: Изучение HTTP-методов и их применения для реализации CRUD-операций. Создание REST-подобного API с хранением данных в памяти сервера.

Цель работы: Освоить обработку различных HTTP-методов (GET, POST, PUT, DELETE) в Express/Flask. Научиться реализовывать CRUD-операции над коллекцией объектов, хранящейся в памяти сервера, а также возвращать корректные HTTP-коды ответов (200, 201, 404).

Теоретическое обоснование

Что такое CRUD

CRUD - это акроним, обозначающий четыре базовые операции над данными:

Операция	Расшифровка	HTTP-метод	SQL-аналог
Create	Создание	POST	INSERT
Read	Чтение	GET	SELECT
Update	Обновление	PUT / PATCH	UPDATE
Delete	Удаление	DELETE	DELETE

Аналогия: Представьте библиотеку. CRUD - это:

Create - принести новую книгу и поставить на полку.

Read - найти и прочитать книгу.

Update - заменить старую книгу на новое издание.

Delete - выбросить книгу.

HTTP-методы подробнее

GET - получение данных

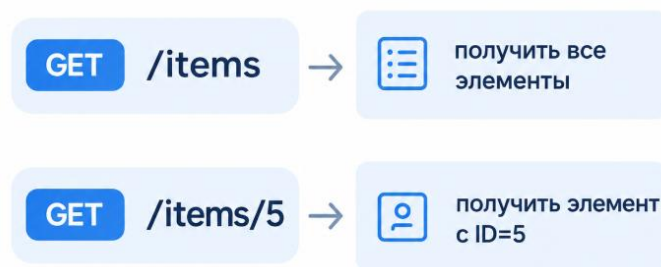
Назначение: запросить данные с сервера.

Тело запроса: отсутствует.

Идемпотентность: да (повторный запрос не меняет состояние).

Безопасность: да (не изменяет данные).

Коды ответов: 200 OK, 404 Not Found.



POST - создание данных

Назначение: создать новый ресурс.

Тело запроса: содержит данные нового ресурса.

Идемпотентность: нет (два одинаковых запроса создадут два ресурса).

Безопасность: нет (изменяет данные).

Коды ответов: 201 Created, 400 Bad Request.



PUT - полное обновление

Назначение: полностью заменить существующий ресурс.

Тело запроса: содержит новые данные ресурса.

Идемпотентность: да (повторный запрос даёт тот же результат).

Безопасность: нет (изменяет данные).

Коды ответов: 200 OK, 404 Not Found.

PUT /items/5 → заменить элемент с ID=5

DELETE - удаление

Назначение: удалить ресурс.

Тело запроса: отсутствует.

Идемпотентность: да (повторное удаление не меняет результат).

Безопасность: нет (изменяет данные).

Коды ответов: 200 OK, 204 No Content, 404 Not Found.

DELETE /items/5 → удалить элемент с ID=5

Коды состояния HTTP

Код	Название	Когда используется
200	OK	Успешный GET, PUT, DELETE
201	Created	Успешный POST (ресурс создан)
204	No Content	Успешный DELETE (нет тела ответа)
400	Bad Request	Ошибка в данных запроса
404	Not Found	Ресурс не найден
500	Internal Server Error	Ошибка на сервере

Хранение данных в памяти

В этой работе мы не используем базу данных. Данные хранятся в массиве (или объекте) в оперативной памяти сервера.

Преимущества:

- Простота
- Не нужна установка БД
- Быстрый доступ

Недостатки:

- Данные теряются при перезапуске сервера
- Не подходит для production
- Пример хранения:

```
1
2  let items = [
3    { id: 1, name: 'Товар 1', price: 100 },
4    { id: 2, name: 'Товар 2', price: 200 }
5  ];
6
```

Пример выполнения

Шаг 1. Подготовка проекта

Используйте проект из лабораторной работы №1 или создайте новый:

Работаем в консоли bash или терминал VC

```
mkdir lab2-crud
cd lab2-crud
npm init -y
npm install express
npm install --save-dev nodemon
```

```
Problems Output Debug Console Terminal

Сервер запущен на http://localhost:3000
[2026-09-02T20:56:00.031Z] GET /api/status

* History restored

Microsoft Windows [Version 10.0.19045.3803]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

D:\lab1-backend>cd..

D:\>mkdir lab2-crud

D:\>cd lab2-crud

D:\lab2-crud>npm init -y
Wrote to D:\lab2-crud\package.json:

{
  "name": "lab2-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

D:\lab2-crud>npm install express

added 68 packages, and audited 69 packages in 3s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\lab2-crud>npm install --save-dev nodemon

added 26 packages, and audited 95 packages in 3s
```

Результат работы команд

Шаг 2. Настройка package.json

Добавьте скрипты:

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```

Шаг 3. Создание сервера с CRUD-операциями

Создайте файл app.js:

```
const express = require('express');
```

```
const app = express();
const port = 3000;

// Middleware для парсинга JSON из тела запроса
app.use(express.json());

// Middleware для логирования запросов
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// ===== ХРАНИЛИЩЕ ДАННЫХ В ПАМЯТИ =====
let items = [
  { id: 1, name: 'Товар 1', price: 100, quantity: 5 },
  { id: 2, name: 'Товар 2', price: 200, quantity: 3 },
  { id: 3, name: 'Товар 3', price: 300, quantity: 10 }
];

// Счётчик для генерации новых ID
let nextId = 4;

// ===== CRUD-ОПЕРАЦИИ =====

// GET /items — получить все элементы
app.get('/items', (req, res) => {
  res.json({
    count: items.length,
    items: items
  });
});

// GET /items/:id — получить один элемент
app.get('/items/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const item = items.find(i => i.id === id);

  if (!item) {
    return res.status(404).json({ error: 'Элемент не найден' });
  }

  res.json(item);
});

// POST /items — создать новый элемент
app.post('/items', (req, res) => {
  const { name, price, quantity } = req.body;

  // Валидация: обязательные поля
  if (!name || price === undefined) {
    return res.status(400).json({
```

```

        error: 'Поля name и price обязательны'
    });
}

// Создаём новый элемент
const newItem = {
    id: nextId++,
    name: name,
    price: price,
    quantity: quantity || 0
};

items.push(newItem);

// 201 Created – ресурс создан
res.status(201).json(newItem);
});

// PUT /items/:id – обновить элемент
app.put('/items/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = items.findIndex(i => i.id === id);

    if (index === -1) {
        return res.status(404).json({ error: 'Элемент не найден' });
    }

    const { name, price, quantity } = req.body;

    // Полное обновление
    items[index] = {
        id: id,
        name: name || items[index].name,
        price: price !== undefined ? price : items[index].price,
        quantity: quantity !== undefined ? quantity : items[index].quantity
    };

    res.json(items[index]);
});

// DELETE /items/:id – удалить элемент
app.delete('/items/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = items.findIndex(i => i.id === id);

    if (index === -1) {
        return res.status(404).json({ error: 'Элемент не найден' });
    }

    const deletedItem = items.splice(index, 1)[0];

```

```

// 200 OK с удалённым элементом
res.json({
  message: 'Элемент удалён',
  deleted: deletedItem
});
});

// ===== ОБРАБОТКА 404 =====
app.use((req, res) => {
  res.status(404).json({ error: 'Маршрут не найден' });
});

// ===== ЗАПУСК СЕРВЕРА =====
app.listen(port, () => {
  console.log(`Сервер запущен на http://localhost:${port}`);
});

```

Шаг 4. Запуск сервера

bash

npm run dev

Шаг 5. Проверка работы

GET - получить все элементы

http://localhost:3000/items

Результат:

```

{
  "count": 3,
  "items": [
    { "id": 1, "name": "Товар 1", "price": 100, "quantity": 5 },
    { "id": 2, "name": "Товар 2", "price": 200, "quantity": 3 },
    { "id": 3, "name": "Товар 3", "price": 300, "quantity": 10 }
  ]
}

```

GET - получить один элемент

http://localhost:3000/items/2

Результат json:

```
{ "id": 2, "name": "Товар 2", "price": 200, "quantity": 3 }
```

POST - создать новый элемент

Через Postman / Thunder Client:

POST http://localhost:3000/items

Content-Type: application/json

```
{  
  "name": "Новый товар",  
  "price": 500,  
  "quantity": 7  
}
```

Результат (201 Created):

json

```
{ "id": 4, "name": "Новый товар", "price": 500, "quantity": 7 }
```

PUT — обновить элемент

text

PUT http://localhost:3000/items/1

Content-Type: application/json

```
{  
  "name": "Обновлённый товар",  
  "price": 150,  
  "quantity": 20  
}
```

Результат (200 OK):

json

```
{ "id": 1, "name": "Обновлённый товар", "price": 150, "quantity": 20 }
```

DELETE - удалить элемент

DELETE http://localhost:3000/items/3

Результат (200 OK):

json

```
{  
  "message": "Элемент удалён",  
  "deleted": { "id": 3, "name": "Товар 3", "price": 300, "quantity": 10 }  
}
```

Индивидуальные задания

Комплект 1. Базовый уровень

Требования: Реализовать CRUD-операции для одной сущности.

Обязательные эндпоинты:

GET /items - список всех элементов

GET /items/:id - один элемент

POST /items - создание

PUT /items/:id - обновление

DELETE /items/:id - удаление

Обработка ошибок: 404 при отсутствии элемента.

Вариант	Сущность	Поля
1	Задачи (tasks)	id, title, done
2	Пользователи (users)	id, name, email
3	Товары (products)	id, name, price
4	Книги (books)	id, title, author
5	Фильмы (movies)	id, title, year

Вариант	Сущность	Поля
6	Студенты (students)	id, name, group
7	Города (cities)	id, name, population
8	Сотрудники (employees)	id, name, position
9	Курсы (courses)	id, title, duration
10	Автомобили (cars)	id, brand, model
11	Рецепты (recipes)	id, title, time
12	Игры (games)	id, title, genre
13	Отели (hotels)	id, name, stars
14	Рестораны (restaurants)	id, name, cuisine
15	Песни (songs)	id, title, artist

Комплект 2. Средний уровень

Требования: Всё из базового уровня + дополнительные функции:

Валидация входных данных:

Проверка обязательных полей (400 Bad Request).

Проверка типов (например, price - число).

Проверка допустимых значений.

Дополнительные эндпоинты:

GET /items?search=... - поиск по названию.

GET /items?sort=field&order=asc|desc - сортировка.

GET /items?page=1&limit=10 - пагинация.

Правильные коды ответов:

201 Created при POST.

204 No Content при DELETE (без тела ответа).

400 Bad Request при неверных данных.

Вариант	Сущность	Дополнительные поля	Поиск по
1	Задачи	priority, deadline	title
2	Пользователи	age, city	name
3	Товары	category, stock	name
4	Книги	year, genre	title
5	Фильмы	director, rating	title
6	Студенты	course, avg_grade	name
7	Города	country, area	name
8	Сотрудники	salary, department	name
9	Курсы	level, price	title
10	Автомобили	year, price	brand
11	Рецепты	ingredients, difficulty	title
12	Игры	platform, rating	title
13	Отели	city, price_per_night	name
14	Рестораны	address, rating	name
15	Песни	album, duration	title

Комплект 3. Продвинутый уровень

Требования: Всё из среднего уровня + дополнительные функции:

Частичное обновление (PATCH):

PATCH /items/:id - обновление только переданных полей.

Массовые операции:

DELETE /items - удалить все элементы.

POST /items/bulk - создать несколько элементов за раз.

Дополнительные эндпоинты:

GET /items/stats - статистика (количество, средняя цена).

GET /items/:id/related - связанные элементы.

Логирование в файл:

Запись всех запросов в файл access.log.

Обработка ошибок 500:

Глобальный обработчик ошибок.

Вариант	Сущность	Статистика	Связи
1	Задачи	Количество выполненных	Задачи по приоритету
2	Пользователи	Средний возраст	По городу
3	Товары	Средняя цена	По категории
4	Книги	Количество по жанрам	По автору
5	Фильмы	Средний рейтинг	По режиссёру
6	Студенты	Средний балл	По группе
7	Города	Общее население	По стране

Вариант	Сущность	Статистика	Связи
8	Сотрудники	Средняя зарплата	По отделу
9	Курсы	Количество по уровням	По уровню
10	Автомобили	Средняя цена	По марке
11	Рецепты	Среднее время	По сложности
12	Игры	Средний рейтинг	По платформе
13	Отели	Средняя цена	По городу
14	Рестораны	Средний рейтинг	По кухне
15	Песни	Общая длительность	По исполнителю

Контрольные вопросы

Базовый уровень

1. Что такое CRUD? Расшифруйте каждую букву.
2. Какие HTTP-методы соответствуют операциям CRUD?
3. Что такое идемпотентность HTTP-методов? Какие методы идемпотентны?
4. Какой код ответа возвращается при успешном создании ресурса (POST)?
5. Какой код ответа возвращается при успешном удалении ресурса?
6. Что такое middleware в Express? Для чего используется `express.json()`?
7. Как получить параметры из тела POST-запроса в Express?

8. Как получить параметр из URL (например, ID)?
9. Как найти элемент в массиве по ID?
10. Какой код ответа возвращается, если ресурс не найден?
11. Как отправить JSON-ответ в Express?
12. Как вернуть статус-код 201 в Express?
13. Что произойдёт с данными в памяти при перезапуске сервера?
14. Как добавить новый элемент в массив?
15. Как удалить элемент из массива?

Средний уровень

1. В чём разница между PUT и PATCH?
2. Как реализовать поиск по коллекции (query-параметр search)?
3. Как реализовать сортировку по полю?
4. Как реализовать пагинацию (постраничный вывод)?
5. Как валидировать входные данные (например, что price — число)?
6. Какие коды ответов используются при ошибках валидации?
7. Как вернуть 204 No Content при DELETE?
8. В чём разница между res.json() и res.send()?
9. Как избежать дублирования ID при создании новых элементов?
10. Что такое express.json() и почему без него req.body пустой?

Продвинутый уровень

1. Как реализовать частичное обновление (PATCH)?
2. Как реализовать массовое удаление элементов?
3. Как реализовать массовое создание элементов?
4. Как реализовать глобальный обработчик ошибок?
5. Как логировать запросы в файл?
6. Что такое REST API и какие принципы лежат в его основе?
7. Как организовать структуру проекта для масштабируемого CRUD API?
8. Какие существуют стратегии генерации уникальных ID?
9. Как защитить API от слишком больших запросов?

10.Как реализовать связь между сущностями (например, задачи и пользователи)?

Требования к отчёту

Отчёт должен быть представлен в формате Markdown и содержать:

Титульный лист - название работы, ФИО, группа, вариант.

Цель работы.

Теоретическое обоснование - краткое изложение CRUD, HTTP-методов, кодов состояния.

Выполнение практического примера - код сервера, скриншоты всех запросов через Postman/Thunder Client/браузер.

Выполнение индивидуального задания - код сервера, скриншоты всех эндпоинтов, включая обработку ошибок.

Ответы на контрольные вопросы - не менее 5 по своему уровню.

Вывод - что было сделано, что нового узнали, с какими трудностями столкнулись.

Список использованных источников.

Критерии оценивания

Критерий	Базовый (max 30)	Средний (max 45)	Продвинутый (max 60)
GET-эндпоинты (список и один элемент)	10	5	5
POST-эндпоинт (создание)	5	5	5
PUT-эндпоинт (обновление)	5	5	5
DELETE-эндпоинт (удаление)	5	5	5

Критерий	Базовый (max 30)	Средний (max 45)	Продвинутый (max 60)
Обработка ошибки 404	5	5	5
Валидация входных данных (400)	—	10	5
Поиск, сортировка, пагинация	—	10	5
PATCH (частичное обновление)	—	—	5
Массовые операции	—	—	5
Дополнительные эндпоинты (stats, related)	—	—	5
Логирование в файл	—	—	5
Глобальный обработчик ошибок	—	—	5
Итого	30	45	60

Шкала перевода баллов в оценку:

Оценка	Базовый (30)	Средний (45)	Продвинутый (60)
5 (отлично)	—	—	54-60
4 (хорошо)	—	40-45	46-53

Оценка	Базовый (30)	Средний (45)	Продвинутый (60)
3 (удовлетворительно)	24-30	34-39	38-45
2 (неудовлетворительно)	0-23	0-33	0-37

Рекомендуемые источники

Express - Routing - <https://expressjs.com/en/guide/routing.html>

Express - Request и Response - <https://expressjs.com/en/4x/api.html>

HTTP-методы

(MDN) - <https://developer.mozilla.org/ru/docs/Web/HTTP/Methods>

Коды состояния HTTP (MDN)

- <https://developer.mozilla.org/ru/docs/Web/HTTP/Status>

REST API Tutorial - <https://restfulapi.net/>

Postman Learning Center - <https://learning.postman.com/>

Thunder Client - <https://www.thunderclient.com/>